

# Exercise: Genetic Algorithm

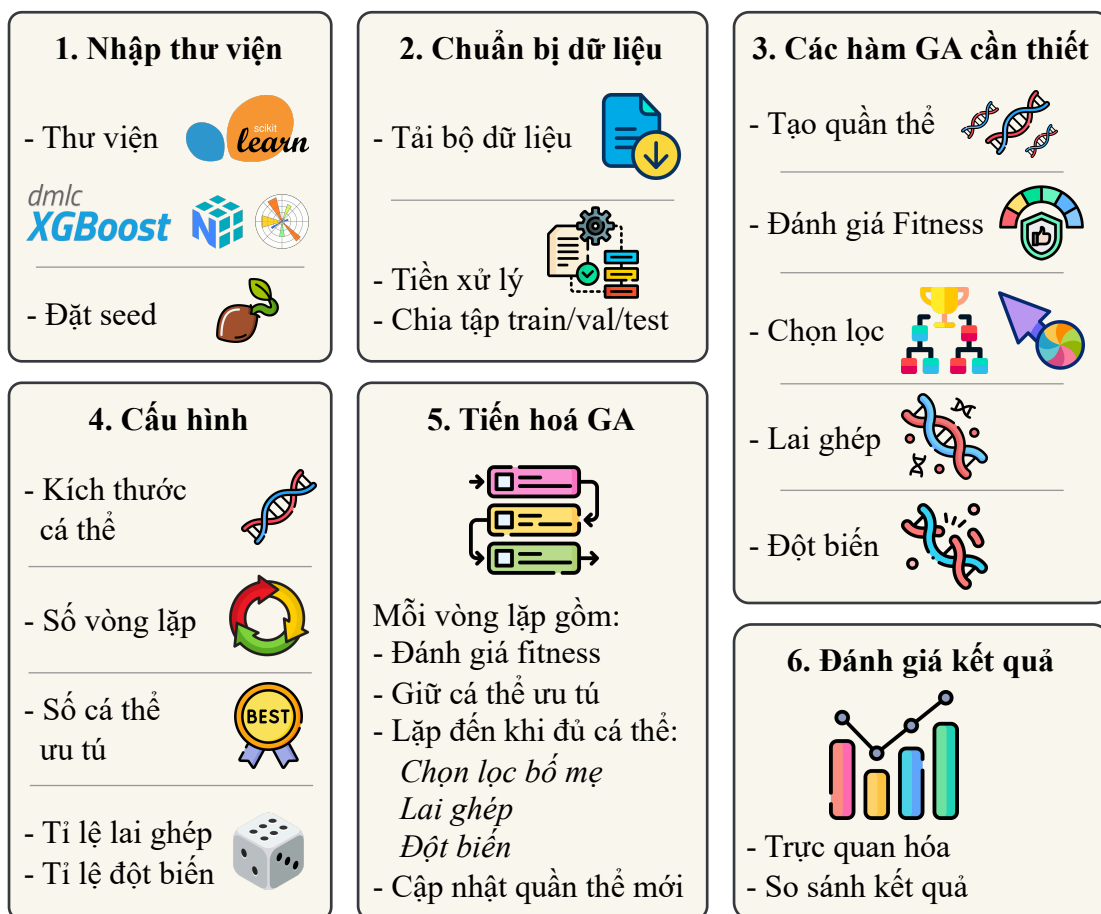
Dương Đình Thắng

Nguyễn Anh Khôi

Đinh Quang Vinh

## I. Giới thiệu

Trong bài này, chúng ta sẽ thực hành lập trình về nội dung **Genetic Algorithm (GA)**. GA là phương pháp tối ưu hoá theo quần thể lời giải, trong đó mỗi thể hệ sẽ thực hiện các bước tính toán gồm: *fitness* (tạm dịch: đánh giá), *selection* (tạm dịch: chọn lọc), *crossover* (tạm dịch: lai ghép) và *mutation* (tạm dịch: đột biến). Qua loạt bài tập, chúng ta củng cố lại các khái niệm cốt lõi trong GA đã học và kết nối tất cả thành một pipeline cài đặt code hoàn chỉnh để giải một số bài toán học máy cơ bản.



Hình 1: Các bước sẽ thực hiện trong bài tập Genetic Algorithm (GA).

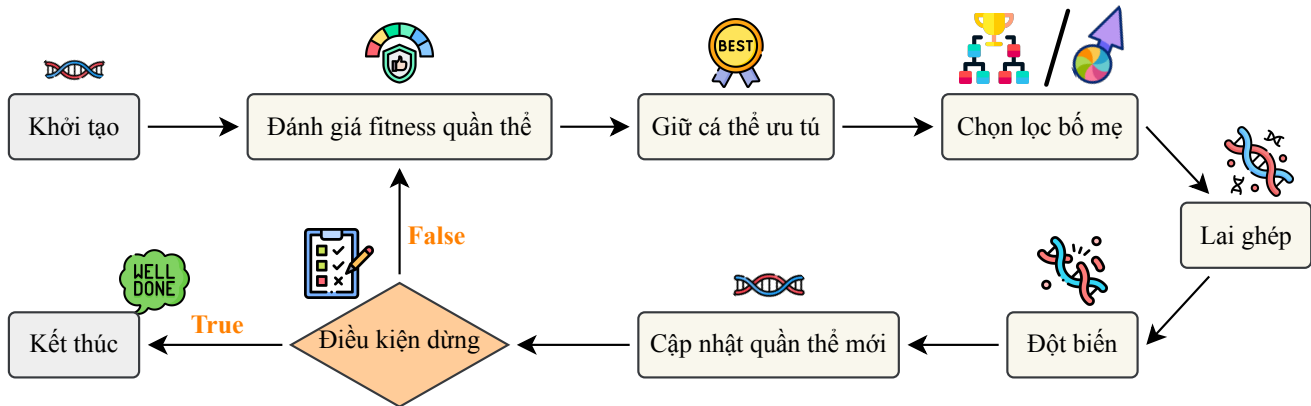
# Mục lục

|             |                                             |           |
|-------------|---------------------------------------------|-----------|
| <b>I.</b>   | <b>Giới thiệu . . . . .</b>                 | <b>1</b>  |
| <b>II.</b>  | <b>Thực hành . . . . .</b>                  | <b>5</b>  |
| II.1.       | Bài tập 1: OneMax và Trap Fitness . . . . . | 5         |
| II.2.       | Bài tập 2: Linear Regression . . . . .      | 8         |
| II.3.       | Bài tập 3: Text Classification . . . . .    | 10        |
| <b>III.</b> | <b>Câu hỏi trắc nghiệm . . . . .</b>        | <b>13</b> |
|             | <b>Phụ lục . . . . .</b>                    | <b>17</b> |

Theo đó, nội dung bài tập lần này sẽ gồm có ba bài tập thực hành lớn và các câu trắc nghiệm đi kèm. Tóm tắt nội dung của các bài thực hành được thể hiện qua bảng sau:

| Tổng quan 3 bài thực hành |                                                                                                           |                                                                |                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Bài                       | Mục tiêu                                                                                                  | Biểu diễn                                                      | Hàm Fitness                                                                                       |
| 1                         | Tối đa hoá số bit 1.                                                                                      | Chuỗi nhị phân                                                 | Tổng số bit 1 hoặc <i>trap</i> theo block                                                         |
| 2                         | Tối ưu vector trọng số hồi quy tuyến tính để giảm MSE.                                                    | Vector thực $\mathbf{w} \in \mathbb{R}^d$                      | $\text{fitness} = \frac{1}{\text{MSE}_{\text{val}} + \varepsilon}$<br>( $\varepsilon = 10^{-8}$ ) |
| 3a                        | Chọn tập đặc trưng TF-IDF (1000 chiều) cho phân loại đa lớp, tối ưu <i>macro-F1</i> và phạt số đặc trưng. | biểu diễn nhị phân $\mathbf{m} \in \{0, 1\}^{1000}$            | $\text{fitness} = \text{F1}_{\text{macro, val}} - \lambda \ \mathbf{m}\ _0$                       |
| 3b                        | Dùng GA chọn tham số tối ưu cho <i>XGBoost</i> .                                                          | Nhiệm sắc thể tham số hỗn hợp $\theta = [\text{real/int/cat}]$ | $\text{fitness} = \text{F1}_{\text{macro, val}}$ (hoặc điểm CV trên tập validation)               |

Mặc dù bối cảnh ứng dụng khác nhau, song cả ba bài đều tuân theo cùng một chu trình của **GA**: khởi tạo quần thể, *đánh giá fitness*, *chọn lọc*, *lai ghép*, *đột biến* (kèm *elitism*) và cập nhật thể hệ. Dưới đây là *pipeline* tổng quát các bước cài đặt GA trong Python:



Hình 2: Pipeline tổng quan các bước tính toán trong GA.

## 1. Nhập thư viện

- Tải các gói/phụ thuộc cho sinh ngẫu nhiên, xử lý mảng, trực quan hoá và đánh giá.
- Định nghĩa `set_random_seed(seed)` để cố định nguồn ngẫu nhiên cho toàn bộ thí nghiệm.

## 2. Chuẩn bị dữ liệu

- Xác định nguồn dữ liệu và mục tiêu tối ưu.
- Tiền xử lý theo yêu cầu: chuẩn hoá/thang đo, mã hoá đặc trưng, tách train/validation/test hoặc cross-validation; mọi biến đổi được *fit* trên train và áp dụng nhất quán lên các tập còn lại.

## 3. Các hàm/toán tử GA cần thiết

- Khởi tạo cá thể/quần thể: `create_individual()`, `init_population()` (nhị phân, số thực hoặc hỗn hợp tùy bài).
- Đánh giá độ thích nghi: `compute_fitness()` (định nghĩa phù hợp với mục tiêu; có thể cần dịch/chuẩn hoá miền giá trị).
- Chọn lọc: `selection_operator()` (ví dụ: tournament, proportional, rank, stochastic universal sampling).
- Lai ghép và đột biến: `crossover_operator()` (một điểm, hai điểm, đồng nhất, hoặc kết hợp tuyến tính cho biến thực); `mutation_operator()` (lật bit, nhiễu Gauss, hoán vị, kẹp biên/chiều miền khả thi khi cần).

## 4. Cấu hình thực nghiệm

- Thiết lập siêu tham số: `pop_size`, `generations`, `elitism`, `crossover_rate`, `mutation_rate` và tham số riêng của toán tử (ví dụ `tournament_size`).
- Xác định các kịch bản so sánh (thay đổi selection, tỷ lệ crossover/mutation, lịch mutation, v.v.) để phân tích độ nhạy.

## 5. Tiến hoá GA

- Khởi tạo quần thể và đánh giá fitness ban đầu.
- Lặp theo thể hệ:
  - (a) Bảo toàn elitism (sao chép một số cá thể tốt nhất).
  - (b) Chọn cặp cha mẹ bằng `selection_operator()`.
  - (c) Tạo con bằng `crossover_operator()` và áp dụng `mutation_operator()`.
  - (d) Đánh giá fitness thể hệ con, hình thành quần thể mới (gộp elite).
  - (e) Ghi nhật ký (ít nhất best/avg fitness); áp dụng dừng sớm khi thỏa tiêu chí (ngưỡng, patience, hội tụ).
- Trả về cá thể tốt nhất và lịch sử fitness.

## 6. Đánh giá kết quả

- Khoá cấu hình theo cá thể tốt nhất trên validation và báo cáo trên test/hold-out/CV.
- Trình bày chỉ số phù hợp mục tiêu (ví dụ: sai số, độ chính xác, F-measure, mục tiêu đa mục tiêu nếu có).
- Trực quan hoá: đường hội tụ fitness, phân tích ổn định/độ nhạy, và các biểu đồ hỗ trợ giải thích tùy bài toán.

Thông qua các bước thực hiện cơ bản của GA được mô tả ở trên, chúng ta sẽ tiến hành ứng dụng vào các bài tập ở phần sau.

## II. Thực hành

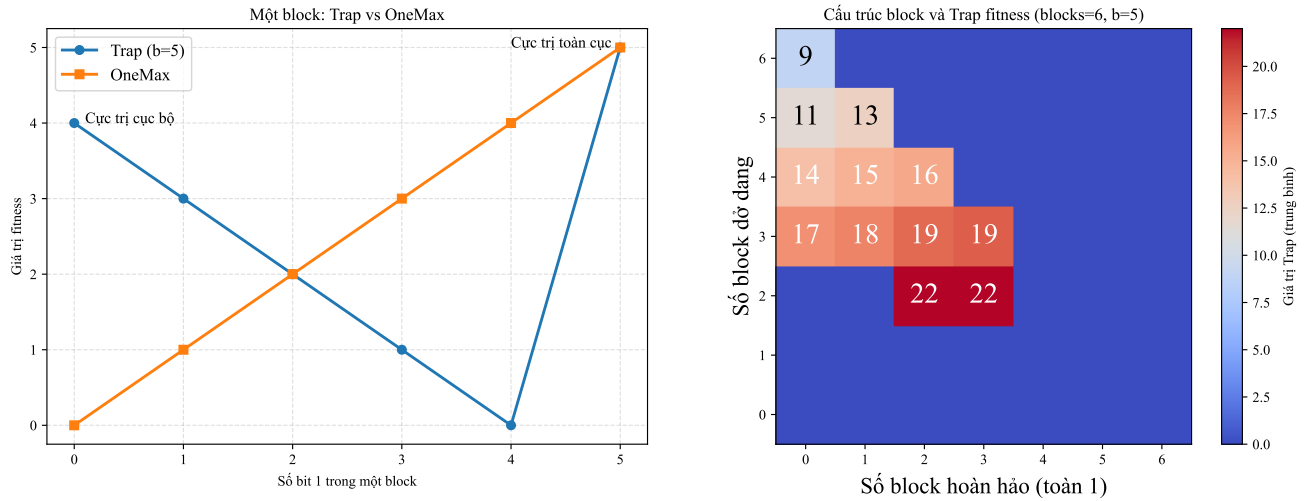
### II.1. Bài tập 1: OneMax và Trap Fitness

#### II.1.1. Phát biểu bài toán

##### Nội dung

Triển khai **GA** trên không gian nhị phân và so sánh hai cơ chế chọn lọc: Tournament Selection và Proportional Selection (Roulette-Wheel). Hai hàm fitness sử dụng là OneMax và Trap. Sau khi hoàn thành chương trình, ta quan sát đường hội tụ và làm rõ vai trò của chọn lọc, đột biến và elitism trong việc cân bằng giữa khai thác và khám phá.

**Trap** là một hàm fitness được thiết kế để tạo ra bẫy cục bộ. Ở mỗi block độ dài  $b$  (thường dùng  $b = 5$ ), cá thể chỉ nhận điểm tối đa khi toàn bộ bit trong block đều bằng 1; nếu còn thiếu bất kỳ bit 1 nào thì điểm bị phạt theo mức độ “dở dang”. Cấu trúc thưởng-phạt này khiến quần thể dễ mắc kẹt tại các nghiệm cục bộ dù còn xa tối ưu toàn cục, do các thay đổi nhỏ xung quanh nghiệm cục bộ làm giảm fitness. So với **OneMax** - nơi fitness tăng đều theo số bit 1 - Trap khó hơn đáng kể vì các bước cải thiện cục bộ không dẫn tới tiến bộ toàn cục và dễ khiến lựa chọn tham lam hoặc áp lực chọn lọc cao hội tụ sớm.



Hình 3: So sánh landscape giữa OneMax và Trap. (Trái) Một block đơn ( $b = 5$ ); (Phải) Cấu trúc nhiều block ( $n_{\text{blocks}} = 6, b = 5$ ).

Hình 3 cho thấy rõ cách mà hàm **Trap** tạo ra **bẫy cục bộ** trong không gian tìm kiếm:

- **Biểu đồ bên trái:** minh họa hình dạng của hàm fitness với một block gồm 5 bit. Với OneMax (đường cam), giá trị fitness tăng tuyến tính theo số bit 1, được cho bởi

$$f(\mathbf{x}) = f_{\text{ONEMAX}}(u), \quad u = \sum_{i=1}^5 x_i,$$

$$f_{\text{ONEMAX}}(u) = u \quad (\mathbf{x} \in \{0, 1\}^5).$$

Nghĩa là càng nhiều bit 1 thì fitness càng cao, dẫn thẳng tới nghiệm toàn cục tại  $u = 5$ . Trái lại, với **Trap** (đường xanh), chỉ khi toàn bộ bit trong block đều bằng 1 mới đạt điểm tối đa; các trạng thái “dở dang” bị phạt. Ví dụ, với block  $b = 5$ :

$$f(\mathbf{x}) = f_{\text{TRAP}}(u), \quad u = \sum_{i=1}^5 x_i,$$

$$f_{\text{TRAP}}(u) = \begin{cases} 5, & \text{nếu } u = 5, \\ 4 - u, & \text{nếu } u < 5. \end{cases}$$

Các chuỗi gần đúng (như  $u = 4$ ) vẫn bị giảm điểm, tạo một **local optimum** tại  $u = 0$ . Do đó, quần thể tiến hoá tuyến tính dễ “rơi vào bẫy” này và khó vượt ngưỡng trung gian để đạt tối ưu toàn cục.

- **Biểu đồ bên phải:** mở rộng thêm nhiều block ( $n_{\text{blocks}} = 6$ , mỗi block dài  $b = 5$ ). Trục hoành thể hiện số block hoàn hảo (chứa toàn bit 1), trục tung là số block chưa hoàn thiện. Mỗi ô trong heatmap biểu diễn **giá trị fitness trung bình** của các nhiễm sắc thể có cấu trúc tương ứng. Màu đỏ thể hiện vùng fitness cao, xanh là thấp. Dễ thấy các tổ hợp có vài block hoàn hảo xen lẫn block dở dang (góc giữa bên phải) lại bị phạt nặng, dù tổng số bit 1 không nhỏ để minh họa hiện tượng “đánh lừa” của trap: *những tiến bộ cục bộ không dẫn đến lợi ích toàn cục*.

### II.1.2. Mục tiêu của bài tập

- Cài đặt và chạy GA cơ bản trên không gian nhị phân, tương ứng với pipeline được mô tả ở Hình 2.
- So sánh hành vi hội tụ giữa hai cơ chế chọn lọc: **Tournament** (áp lực chọn cao) và **Proportional** (xác suất tỉ lệ với fitness).
- Quan sát sự khác biệt về hình dạng của hàm fitness với **OneMax** và **Trap**.
- Đánh giá ảnh hưởng của **elitism** và **mutation rate** đến khả năng thoát khỏi cực trị cục bộ.

### II.1.3. Yêu cầu

Dựa vào file code **[Hint]\_Exercise\_01** trong Phụ lục, các bạn hãy hoàn thiện các phần còn thiếu để chạy trọn vẹn pipeline của bài. Cụ thể:

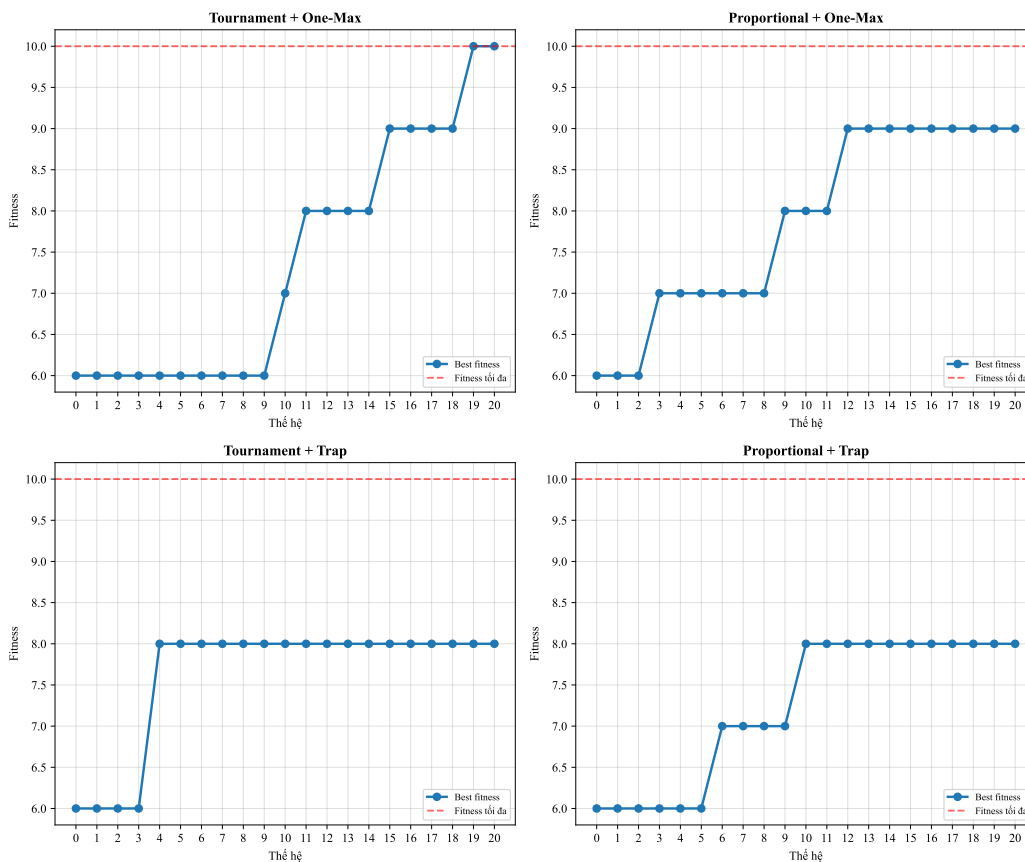
- Fitness: hỗ trợ OneMax (đếm tổng bit 1) và Trap (chia block 5 bit, chỉ thưởng khi cả block đều là 1, phạt trạng thái chưa hoàn chỉnh).
- Toán tử crossover/mutation/elitism: hiện thực `uniform_crossover`, `bitflip_mutation` và cơ chế elitism.
- Cơ chế selection: triển khai `tournament` và `proportional` (Roulette–Wheel).

Cấu hình mặc định dùng trong bài:

**Bảng cấu hình siêu tham số - Bài 1**

| Tham số                         | Giá trị |
|---------------------------------|---------|
| Kích thước quần thể (pop_size)  | 5       |
| Số thế hệ (generations)         | 21      |
| Số cá thể ưu tú (elitism)       | 2       |
| Tỉ lệ lai ghép (crossover_rate) | 0.9     |
| Tỉ lệ đột biến (mutation_rate)  | 0.1     |
| Độ dài chuỗi (CHROM_SIZE)       | 10      |
| Seed cố định (seed)             | 42      |

Sau khi hoàn thành phần code trên, các bạn có thể kỳ vọng kết quả thu được như sau:



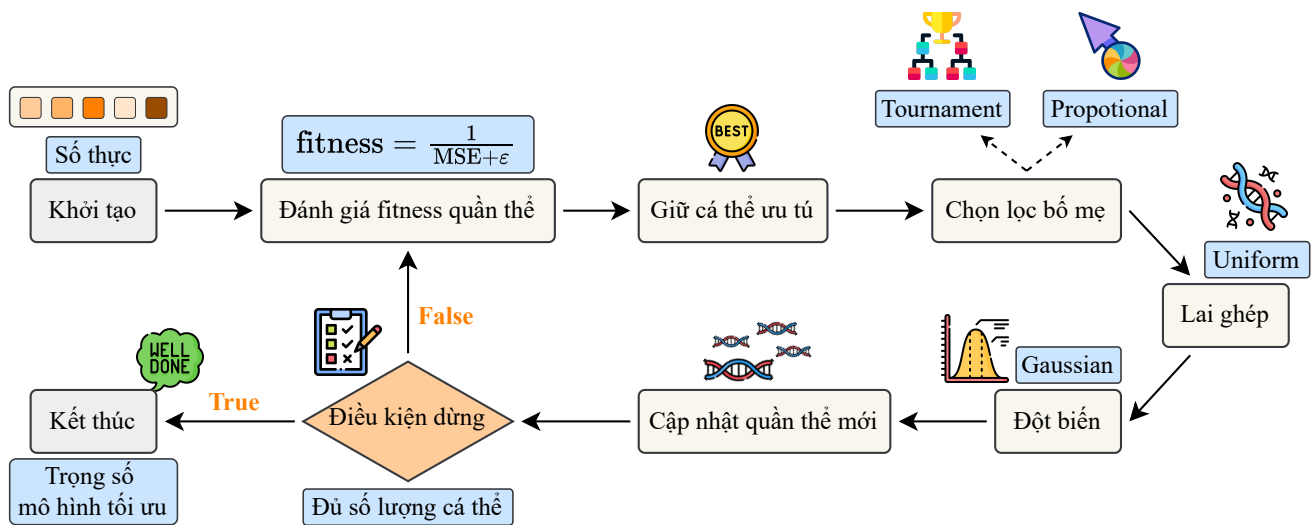
Hình 4: Minh họa việc so sánh fitness đạt được trên hàm OneMax và Trap của hai chiến lược chọn lọc khác nhau.

## II.2. Bài tập 2: Linear Regression

### II.2.1. Phát biểu bài toán

#### Nội dung

Sử dụng **GA** để tìm trọng số tối ưu trong hồi quy tuyến tính trên bộ dữ liệu Diabetes đã chuẩn hoá. Mỗi chromosome là một vector thực  $\theta = [\mathbf{w}; b]$  gồm  $\mathbf{w}$  và  $b$ . Hàm fitness dựa trên Mean Squared Error (MSE), MSE giảm thì fitness tăng. Sau khi hoàn thành chương trình, ta so sánh hai cơ chế chọn lọc Tournament Selection và Proportional Selection, quan sát đường hội tụ, chất lượng dự đoán và ảnh hưởng của tham số đột biến  $\sigma$ .



Hình 5: Minh hoạ bài toán hồi quy tuyến tính được tối ưu bằng GA.

So với Bài 1 (nhị phân), bài này khác ở ba điểm chính để phù hợp với hồi quy tuyến tính: (i) biểu diễn cá thể là vector thực  $\theta = [\mathbf{w}; b]$  thay cho chuỗi bit; (ii) fitness được xây dựng từ MSE trên validation; (iii) đột biến dùng nhiễu Gaussian  $\mathcal{N}(0, \sigma^2)$  thay cho bit-flip. Các thành phần còn lại giữ nguyên tinh thần của Bài 1: chọn lọc (Tournament/Proportional), uniform crossover trên tham số thực, elitism và vòng lặp **GA**. Khuyến nghị chuẩn hoá đặc trưng trước khi tối ưu, báo cáo MSE trên train/validation/test và vẽ đường hội tụ.

### II.2.2. Mục tiêu của bài tập

- Mô hình hoá chromosome thực  $\theta = [\mathbf{w}; b]$  và xây dựng fitness từ MSE.
- Cài đặt và so sánh **Tournament Selection** và **Proportional Selection**.
- Phân tích hội tụ (best-train MSE) và năng lực tổng quát hoá (MSE trên test).

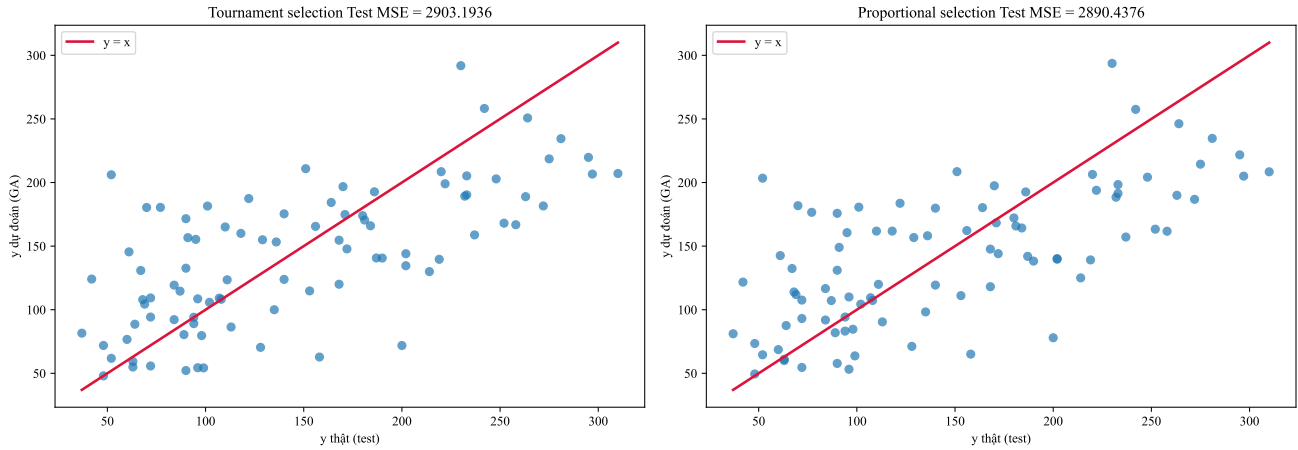


### II.2.3. Yêu cầu

Dựa vào file code [Hint]\_Exercise\_02 trong Phụ lục, hãy hoàn thiện các phần còn thiếu để chạy trọn vẹn pipeline. Cụ thể:

- Fitness/MSE: điền `predict_linear` và `mse_loss` để tính MSE trên dữ liệu đã chuẩn hoá.
- Selection: hoàn thiện `tournament_select` (tham số `tournament_size`) và `proportional_select` (xây probs từ  $1/\text{loss}$ ).
- Crossover/Mutation: hiện thực `uniform_crossover` (biểu diễn 0.5 khi thoả `crossover_rate`) và `gaussian_mutation` (nhiều  $\mathcal{N}(0, \sigma^2)$  áp dụng theo `mutation_rate`).
- Vòng lặp GA: hoàn thiện `evaluate_population` (sắp theo loss tăng dần) và `collect_elites`.

Sau khi hoàn thành cài đặt, các bạn có thể ghi nhận lại kết quả đạt được với GA đối với bài toán Linear Regression trên hai chiến lược chọn lọc:



Hình 6: Minh họa kết quả MSE đạt được trên tập test sau của hai chiến lược chọn lọc khác nhau.

## II.3. Bài tập 3: Text Classification

### II.3.1. Phát biểu bài toán

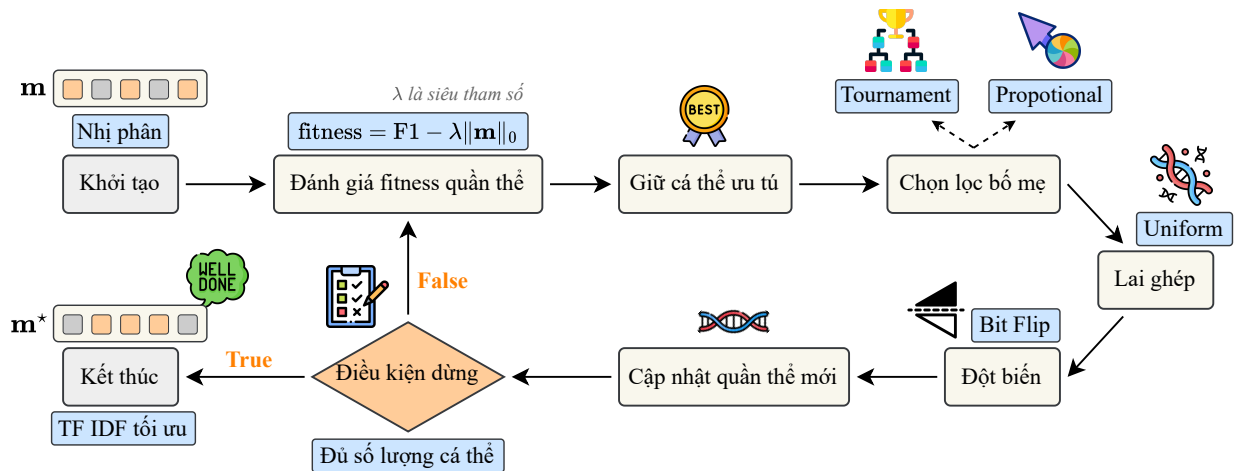
#### Nội dung

Xử lý bài toán phân loại văn bản trên tập 20 `Newsgroups` với bốn chủ đề: `comp.graphics`, `rec.sport.hockey`, `sci.space`, `talk.politics.mideast` với mô hình XGBoost. Dữ liệu văn bản được biểu diễn bằng TF-IDF (tối đa 1000 đặc trưng) và tách train/validation/test như trong file mã nguồn tham khảo. Trong đó, ta ứng dụng GA để giải quyết hai phần sau:

- **3a – GA chọn đặc trưng:** dùng biểu diễn nhị phân (binary mask) trên không gian TF-IDF để tối ưu macro-F1 trên validation, kèm điểm phạt theo số đặc trưng được chọn.
- **3b – GA tinh chỉnh tham số XGBoost:** tối ưu bộ siêu tham số `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `reg_lambda` nhằm cải thiện macro-F1 trên validation, sau đó đánh giá trên test.

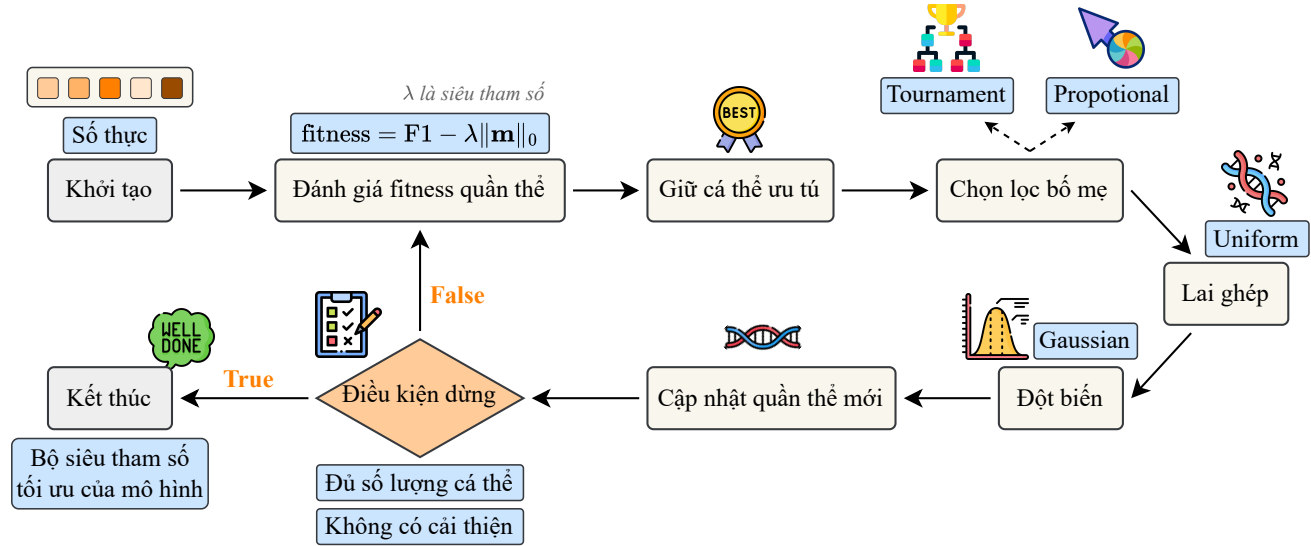
Ở phần bài tập này, khác với hai bài trước, ta sẽ ứng dụng GA vào một bài toán học máy thực tế hơn. Cụ thể, ta sẽ triển khai một bài toán liên quan đến xử lý ngôn ngữ tự nhiên, làm việc với văn bản nên đặc trưng đầu vào được mã hoá TF-IDF có kích thước lớn và thưa. Với hai ngữ cảnh:

1. Sử dụng GA để tìm đặc trưng tối ưu trong việc biểu diễn văn bản.



Hình 7: Minh họa bài toán tìm đặc trưng tối ưu trong việc biểu diễn văn bản bằng GA.

2. Với đặc trưng tối ưu đã tìm được, ta ứng dụng để huấn luyện mô hình phân loại XGBoost, đồng thời ứng dụng GA để tìm bộ siêu tham số tốt nhất cho mô hình.



Hình 8: Minh họa bài toán tìm các siêu tham số tối ưu cho mô hình XGBoost bằng GA.

Fitness ưu tiên chất lượng phân loại (macro-F1 trên validation) và có thể kèm phạt độ thưa để khuyến khích mô hình gọn nhẹ. Hai kiểu chromosome sẽ xuất hiện: biểu diễn nhị phân để chọn đặc trưng (3a) và chuỗi tham số hỗn hợp cho XGBoost (3b). Khi đánh giá cần giữ tỉ lệ lớp ổn định (tách dữ liệu theo kiểu stratified) và báo cáo thêm confusion matrix để quan sát lỗi.

### II.3.2. Mục tiêu của bài tập

- Xây dựng pipeline tiền xử lý văn bản với TF-IDF, tách train/validation/test theo stratified split.
- Dùng **GA** để tối ưu macro-F1: (3a) bằng chọn đặc trưng với biểu diễn nhị phân, (3b) bằng tinh chỉnh siêu tham số XGBoost.
- Phân tích ảnh hưởng của selection, crossover, mutation và elitism lên tốc độ hội tụ và chất lượng mô hình.

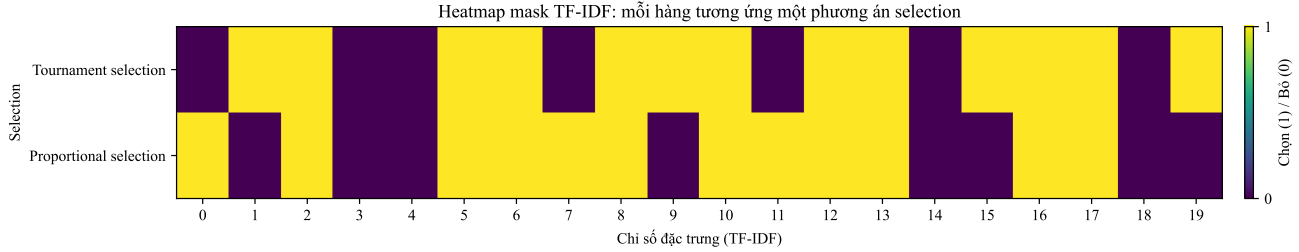
### II.3.3. Yêu cầu

Dựa vào file code [Hint]\_Exercise\_03 trong Phụ lục, hãy hoàn thiện các phần còn thiếu để chạy trọn vẹn hai phần 3a và 3b.

#### 3a – GA chọn đặc trưng TF-IDF

- Biểu diễn: chromosome là biểu diễn nhị phân  $m$  có kích thước bằng số đặc trưng TF-IDF.
- Fitness: `macro_F1_val` trừ đi hệ số phạt theo số đặc trưng được chọn  $\lambda||m||_0$ .
- Toán tử: `uniform_crossover`, `bitflip_mutation`, `elitism`; selection gồm `tournament` và `proportional`.

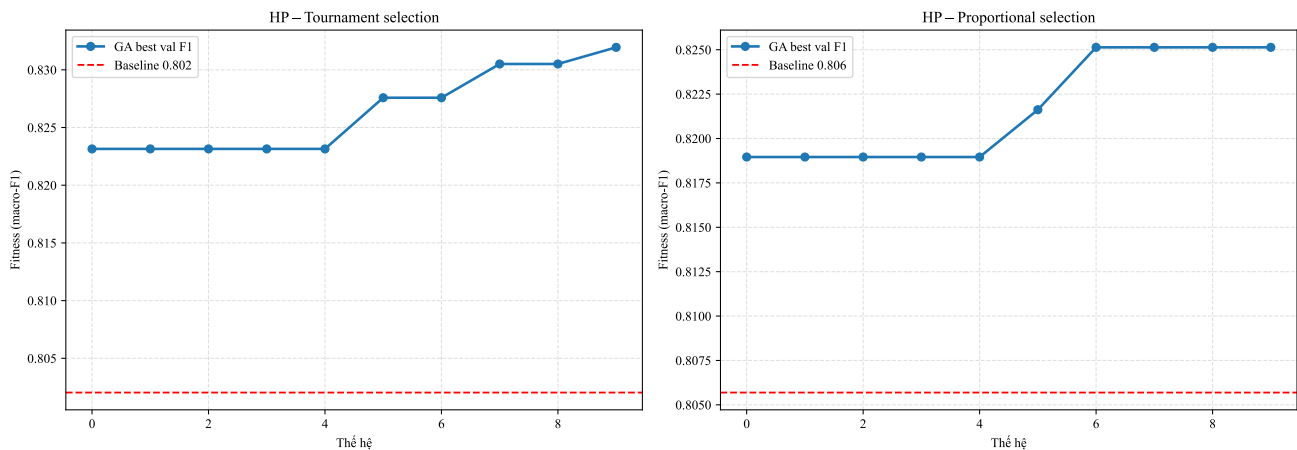
- Đánh giá: huấn luyện mô hình XGBoost trên đặc trưng đã chọn, báo cáo kết quả thông qua độ đo macro-F1 trên validation và test; trực quan hóa bằng đường hội tụ và confusion matrix.



Hình 9: Minh họa trực quan các vị trí/bit trên mask TF-IDF tối ưu sau khi thực hiện GA.

### 3b – GA tinh chỉnh siêu tham số XGBoost

- Biểu diễn: chromosome tham số hỗn hợp [real/int/categorical] gồm các trường như `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `reg_lambda` (xem danh sách chính xác trong mã).
- Fitness: macro-F1 trên validation (có thể dùng trung bình k-fold nếu trong mã có hỗ trợ).
- Toán tử: crossover theo biểu diễn cho tham số số thực/nguyên, hoán đổi giá trị cho tham số rời rạc; mutation kiểu Gaussian cho tham số số thực (kẹp về miền hợp lệ), lật sang lựa chọn khác cho tham số rời rạc; elitism; selection gồm **tournament** và **proportional**.
- Đánh giá: báo cáo macro-F1 validation và test, in bộ tham số tốt nhất, vẽ đường hội tụ.



Hình 10: Minh họa kết quả fitness giữa mô hình baseline với bộ siêu tham số mặc định và mô hình với bộ siêu tham số được lựa chọn bởi GA.

### III. Câu hỏi trắc nghiệm

- Trong một vòng lặp GA điển hình, trình tự thao tác hợp lý nhất là:
  - Chọn cá thể cha mẹ  $\rightarrow$  đánh giá fitness  $\rightarrow$  lai ghép  $\rightarrow$  đột biến  $\rightarrow$  tạo quần thể mới.
  - Lai ghép  $\rightarrow$  đột biến  $\rightarrow$  chọn cá thể cha mẹ  $\rightarrow$  đánh giá fitness  $\rightarrow$  tạo quần thể mới.
  - Đánh giá fitness  $\rightarrow$  chọn cá thể cha mẹ  $\rightarrow$  lai ghép  $\rightarrow$  đột biến  $\rightarrow$  tạo quần thể mới (kèm elitism nếu có).
  - Chuẩn hoá dữ liệu  $\rightarrow$  đánh giá fitness  $\rightarrow$  huấn luyện mô hình sâu  $\rightarrow$  tạo quần thể mới.
- Với hàm Trap theo block kích thước  $b = 5$ : một block đạt điểm 5 nếu có 5 bit bằng 1, ngược lại nhận điểm  $5 - 1 - (\text{số bit } 1)$ . Tính fitness của chuỗi gồm hai block
 
$$[1, 1, 1, 0, 0 \mid 1, 1, 1, 1, 1].$$
  - 4.
  - 5.
  - 6.
  - 7.
- Phát biểu nào sau đây là đúng khi so sánh Tournament Selection và Proportional Selection (Roulette-Wheel)?
  - Tournament luôn đảm bảo chọn cá thể tốt nhất toàn cục ở mọi vòng.
  - Proportional có thể bị chi phối mạnh bởi chênh lệch fitness, dễ mất đa dạng khi chênh lệch quá lớn.
  - Tournament không phụ thuộc vào tham số nào.
  - Proportional không thể chọn cá thể có fitness thấp.
- Chọn phát biểu đúng về mutation trong GA:
  - Bit-flip mutation phù hợp cho biểu diễn nhị phân; Gaussian mutation phù hợp cho vector thực.
  - Gaussian mutation chỉ áp dụng cho bài toán phân loại.
  - Bit-flip mutation làm thay đổi giá trị gene theo phân phối chuẩn.
  - Cả hai loại mutation đều không ảnh hưởng đến khả năng thoát cực trị cục bộ.
- Cho ba cá thể có fitness lần lượt là 2, 3, 5. Dùng Proportional Selection, xác suất chọn cá thể có fitness 3 bằng:
  - 0.20.
  - 0.30.
  - 0.40.
  - 0.50.
- Xét đoạn code sau:

Hàm `proportional_select()`

```

1 def proportional_select(population, fitness_scores):
2     total_fitness = sum(fitness_scores)
3     if total_fitness == 0:
4         return random.choice(population)
5     threshold = random.uniform(0, total_fitness)
6     cumulative = 0.0
7     for individual, fitness in zip(population, fitness_scores):
8         cumulative += fitness
9         if cumulative >= threshold:
10            return individual
11    return population[-1]

```

Hãy cho biết kết quả trả về của hàm khi:

`population = [[1, 0, 1], [0, 1, 0], [1, 1, 1]]`, `fitness_scores = [2, 3, 5]`,

và giá trị ngẫu nhiên lấy từ `random.uniform(0, total_fitness)` giả định bằng 4.7.

- (a) `[1, 0, 1]`.
- (b) `[0, 1, 0]`.
- (c) `[1, 1, 1]`.
- (d) Không xác định vì còn phụ thuộc seed.

7. Xét đoạn code sau:

Hàm `tournament_select()`

```

1 def tournament_select(population, fitness_scores, tournament_size=2):
2     if not population:
3         raise ValueError("Population must contain at least one individual.")
4
5     k = min(tournament_size, len(population))
6     candidate_indices = random.sample(range(len(population)), k=k)
7     best_idx = max(candidate_indices, key=lambda idx: fitness_scores[idx])
8     return population[best_idx]

```

Hãy cho biết hàm trên sẽ trả về giá trị gì trong tình huống sau:

`population = [A, B, C, D]`, `fitness_scores = [3, 7, 5, 2]`, `tournament_size = 3`,

và giả sử `random.sample(range(4), k=3)` trả về `[0, 2, 3]`.

- (a) A.
- (b) B.
- (c) C.
- (d) D.

8. Xét đoạn code sau:

### Chuyển loss thành xác suất chọn trong Proportional Selection

```
1 def selection_probs_from_losses(losses, eps=1e-8):
2     inv = 1.0 / (np.array(losses) + eps)
3     p = inv / np.sum(inv)
4     return p.tolist()
```

Hãy cho biết xác suất chọn (làm tròn 4 chữ số thập phân) khi

`losses = [4.0, 1.0, 5.0]`.

- (a) [0.2000, 0.6000, 0.2000].
- (b) [0.2500, 0.5000, 0.2500].
- (c) [0.1000, 0.8000, 0.1000].
- (d) [0.1724, 0.6897, 0.1379].

9. Xét đoạn code sau:

### Hàm `proportional_select()` cho feature selection

```
1 def proportional_select(population, scores):
2     shifted = np.array(scores) - min(scores)
3     probs = shifted + 1e-6
4     probs /= probs.sum()
5     pick = np.random.choice(len(population), p=probs)
6     return population[pick]
```

Hãy cho biết vector xác suất `probs` (làm tròn 4 chữ số thập phân) khi:

`scores = [0.2, 0.5, 0.5]`.

- (a) [0.2000, 0.5000, 0.3000].
- (b) [0.0000, 0.5000, 0.5000].
- (c) [0.1667, 0.4167, 0.4167].
- (d) [0.2500, 0.3750, 0.3750].

10. Xét đoạn code sau:

### Giải mã chromosome tham số `decode_hp`

```
1 HP_RANGES = {
2     "n_estimators": (10, 100),
3     "max_depth": (2, 10),
4     "learning_rate": (0.01, 0.3),
5     "subsample": (0.5, 1.0),
6     "colsample_bytree": (0.5, 1.0),
7     "reg_lambda": (0.0, 2.0),
```

```

8 }
9 def decode_hp(individual):
10     params = {}
11     for value, key in zip(individual, HP_RANGES):
12         low, high = HP_RANGES[key]
13         val = low + value * (high - low)
14         params[key] = int(val) if key in {"n_estimators", "max_depth"} else float(
15             val)
16     params["n_estimators"] = max(10, int(params["n_estimators"]))
17     params["max_depth"] = max(2, int(params["max_depth"]))
18     params["subsample"] = float(np.clip(params["subsample"], *HP_RANGES["subsample"]
19         ))
20     params["colsample_bytree"] = float(np.clip(params["colsample_bytree"], *
21         HP_RANGES["colsample_bytree"]))
22     return params

```

Giả sử  $\text{individual} = [0, 1, 0.5, 0.5, 0, 1]$ . Bộ tham số giải mã được là gì?

- (a)
  - $\text{n\_estimators} = 10$ .
  - $\text{max\_depth} = 10$ .
  - $\text{learning\_rate} = 0.155$ .
  - $\text{subsample} = 0.75$ .
  - $\text{colsample\_bytree} = 0.50$ .
  - $\text{reg\_lambda} = 2.0$ .
- (b)
  - $\text{n\_estimators} = 55$ .
  - $\text{max\_depth} = 6$ .
  - $\text{learning\_rate} = 0.155$ .
  - $\text{subsample} = 0.50$ .
  - $\text{colsample\_bytree} = 0.75$ .
  - $\text{reg\_lambda} = 1.0$ .
- (c)
  - $\text{n\_estimators} = 10$ .
  - $\text{max\_depth} = 2$ .
  - $\text{learning\_rate} = 0.300$ .
  - $\text{subsample} = 1.00$ .
  - $\text{colsample\_bytree} = 0.50$ .
  - $\text{reg\_lambda} = 2.0$ .
- (d)
  - $\text{n\_estimators} = 100$ .
  - $\text{max\_depth} = 10$ .
  - $\text{learning\_rate} = 0.155$ .
  - $\text{subsample} = 0.75$ .
  - $\text{colsample\_bytree} = 1.00$ .
  - $\text{reg\_lambda} = 0.0$ .



# Phụ lục

1. **Hint:** Các file code gợi ý có thể được tải tại [đây](#).
2. **Solution:** Các file code đáp án của có thể được tải tại [đây](#).
3. **Rubric:**

| Mục  | Kiến thức                                                                                                                                                                                                   | Đánh giá                                                                                                                                                                                                            |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| I.   | <ul style="list-style-type: none"> <li>- Biểu diễn nhị phân, toán tử GA cơ bản (crossover, mutation, elitism).</li> <li>- Khác biệt OneMax vs. Trap; selection: tournament vs. proportional.</li> </ul>     | <ul style="list-style-type: none"> <li>- Hoàn thiện hàm cần thiết; chạy đủ kịch bản; báo cáo đường hội tụ.</li> <li>- Nêu nhận xét ngắn gọn về ảnh hưởng selection, mutation, elitism.</li> </ul>                   |
| II.  | <ul style="list-style-type: none"> <li>- Biểu diễn thực cho tham số hồi quy; dự đoán tuyến tính; MSE làm thước đo.</li> <li>- Gaussian mutation và so sánh selection.</li> </ul>                            | <ul style="list-style-type: none"> <li>- Điền đúng các hàm cốt lõi; vẽ lịch sử MSE; minh họa <math>\hat{y}</math> vs. <math>y</math>.</li> <li>- So sánh ngắn gọn tốc độ/độ chính xác giữa các cấu hình.</li> </ul> |
| III. | <ul style="list-style-type: none"> <li>- TF-IDF và chọn đặc trưng bằng mặt nạ; penalty <math>\lambda \cdot \ \mathbf{m}\ _0</math>.</li> <li>- GA cho tham số XGBoost, macro-F1 (CV/validation).</li> </ul> | <ul style="list-style-type: none"> <li>- Báo cáo F1, số đặc trưng (3a) và bộ tham số tốt nhất (3b).</li> <li>- So sánh với baseline; cung cấp biểu đồ hội tụ/ngắn gọn.</li> </ul>                                   |

4. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

## AIO\_QAs-Verified

🔒 Private group · 1.4K members



Hình 11: Hình ảnh group facebook AIO Q&A.